# C++ Notes: Small Projects & Loops

Guessing Game, For Loops, Power Function, 2D Arrays

**Author: Ba Tung Le**

---

## Learning Objectives

After this section, you will be able to:

- Build a guessing game using `while` / `do-while`.
- Add a guess limit using counters and boolean flags.
- Convert a `while` pattern into a compact `for` loop.
- Use a `for` loop to iterate through an array.
- Build a simple exponent function using repeated multiplication.
- Understand 2D arrays and print them using nested loops.

## Project 1: Building a Guessing Game

**Core idea**

We keep asking the user for guesses until one of these becomes true:

- The user guesses the secret number (win), or
- The user runs out of guesses (lose).

### Version A: Unlimited guesses (basic)

Listing 1: Guessing game (unlimited guesses)

```
int secretNum = 7;
int guess;

while (guess != secretNum) {
  cout << "Enter guess: ";
  cin >> guess;
}

cout << "You win!" << endl;
```

**Why this works**

The loop condition is `guess != secretNum`. As long as the guess is wrong, the condition stays true, so the program keeps prompting.

**Version B: Guess limit (fair game)**

Listing 2: Guessing game with guess limit

```
int secretNum = 7;
int guess;

int guessCount = 0;
int guessLimit = 3;
bool outOfGuesses = false;

while (guess != secretNum && !outOfGuesses) {
  if (guessCount < guessLimit) {
    cout << "Enter guess: ";
    cin >> guess;
    guessCount++;
  } else {
    outOfGuesses = true;
  }
}

if (outOfGuesses) {
  cout << "You lose!" << endl;
} else {
  cout << "You win!" << endl;
}
```

**Common mistake**

If you print `"You win!"` unconditionally after the loop, then the program will claim the user won even when they ran out of guesses. Always check the reason you exited the loop.

**Key pattern**

**Loop until (win) OR (lose)**:

```
while (notCorrect && notOutOfGuesses)
```

That means the loop stops when either:

- guess is correct, or
- outOfGuesses becomes true.

## For Loops

**Core idea**

A `for` loop is a compact version of a common `while` pattern:

- initialize counter
- check condition
- update counter each iteration

### While loop pattern

Listing 3: While loop counting from 1 to 5

```
int index = 1;
while (index <= 5) {
  cout << index << endl;
  index++;
}
```

### Equivalent for loop

Listing 4: Equivalent for loop

```
for (int i = 1; i <= 5; i++) {
  cout << i << endl;
}
```

**Reading the for loop**

- `int i = 1` (start value)
- `i <= 5` (keep looping while true)
- `i++` (run after each loop iteration)

### Loop through an array

Listing 5: Using a for loop to print an array

```
int nums[] = {1, 2, 5, 7, 3};

for (int i = 0; i < 5; i++) {
 cout << nums[i] << endl;
}
```

**Tip**

Array indices start at `0`. If there are 5 elements, valid indices are `0..4`.

## Exponent (Power) Function with a For Loop

**Core idea**

To compute $\text{base}^{\text{power}}$ (for positive integer power), multiply the base repeatedly `power` times.

Listing 6: Power function using repeated multiplication

```
int power(int baseNum, int powNum) {
  int result = 1;

  for (int i = 0; i < powNum; i++) {
    result = result * baseNum;
  }

  return result;
}

int main() {
  cout << power(2, 3) << endl; // 8
  cout << power(4, 2) << endl; // 16
}
```

**Why result starts at 1**

Because 1 is the neutral value for multiplication:

$$1 \cdot \text{base} = \text{base}$$

So the first multiplication gives the correct starting point.

**Limitation**

This simple version assumes `powNum` is non-negative. Handling negative exponents requires floating-point math and division.

## 2D Arrays & Nested Loops

**Core idea**

A 2D array is an array where each element is itself an array. You can think of it as a matrix (rows & columns).

### Create and access a 2D array

Listing 7: 2D array (3 rows x 2 cols)

```
int numberGrid[3][2] = {
  {1, 2},
  {3, 4},
  {5, 6}
};

// Access: numberGrid[row][col]
cout << numberGrid[0][1] << endl; // 2
cout << numberGrid[2][0] << endl; // 5
```

### Print all elements using nested loops

Listing 8: Nested loops to print all elements

```
int numberGrid[3][2] = {
  {1, 2},
  {3, 4},
  {5, 6}
};

for (int i = 0; i < 3; i++) { // rows
  for (int j = 0; j < 2; j++) { // cols
    cout << numberGrid[i][j] << " ";
  }
  cout << endl; // new line after each row
}
```

**How nested loops work**

- Outer loop (`i`) chooses the row.
- Inner loop (`j`) walks through columns in that row.
- After printing a row, print `endl` to move to the next row.

Ba Tung Le
Email: tombn2607@gmail.com